

ClashRoom

Future-Proof Master Spec

Developer continuity blueprint, resilience plan, and AI-independence playbook

Prepared for: Noe Nunez
Company: ClashRoom
Prepared on: March 13, 2026

Purpose: make the system survivable if AI access changes, vendors fail, regulation tightens, or future developers must continue without the original AI collaborator.

Bottom line ClashRoom should be built as a claim-graph product with AI as an optional accelerator, not a single point of failure.

Executive summary

This report future-proofs ClashRoom against the risks most likely to change product development between now and the next three to five years: model vendor concentration, API pricing and access shifts, regulation, app-store privacy pressure, synthetic-media trust problems, and the limits of fully automated verification. The central conclusion is not that AI disappears. It is that AI becomes **more fragmented, more regulated, and more optional**. The winning architecture is therefore a product whose permanent assets are **claims, evidence, relationships, audit logs, and human-operable workflows**. AI should speed up work, not define the product’s existence.

Core recommendation

Build ClashRoom as a durable claim-graph system with provider abstraction, evidence storage, policy logging, manual fallback workflows, and local-model options. If OpenAI, Anthropic, Google, or any other vendor changes terms, another engineering team should still be able to operate the product and replace the AI layer without redesigning the business.

Prediction	Confidence	Why it matters for ClashRoom
AI gets cheaper at the commodity tier but premium reasoning stays expensive	High	Use cheap models for routing/extraction and reserve premium models for edge cases.
Open-weight and self-hosted models become more viable	High	Keep a local fallback path for classification, dedupe, moderation triage, and retrieval.
Regulatory and platform scrutiny increases	High	Need audit logs, human review, disclosures, and explainable moderation actions.
Synthetic media trust becomes a product-level issue, not just a platform issue	High	Track provenance, source quality, and content history as first-class objects.
A single AI vendor becomes a dangerous dependency	Very high	Provider abstraction is mandatory, not optional.

Method note: this is a forecast, not certainty. It does **not** use “all of the world’s data.” It uses a broad but finite set of current public signals, official policies, and technical trend reports. The value here is not perfect prediction; it is designing ClashRoom so the prediction can be wrong and the product still survives.

Part I - What is most likely to change

1. AI will likely become more available, but less simple

Stanford HAI reports that the performance gap between leading closed and open-weight models narrowed sharply by early 2025, while frontier-model performance is converging and smaller models continue to get stronger. At the same time, advanced reasoning methods improve quality but increase cost and latency. The implication is a split market: abundant cheap models for routine tasks, plus expensive premium models for hard cases. ClashRoom should therefore separate workflows into tiers: cheap routing and extraction, medium-cost retrieval and summarization, and premium escalation only for disputes that justify it.

2. Costs should keep falling, but dependence risk remains

Stanford HAI also shows that inference costs have fallen dramatically for models at GPT-3.5-level capability, while official vendor pricing confirms that high-end model pricing still varies materially across model tiers and features. This is good news for volume tasks, but it does not remove dependency risk. A vendor can change product lines, rate limits, safety filters, tooling, or pricing structure even if baseline token costs trend down. Design for price volatility, not just average decline.

3. Regulation and auditability are moving from “nice to have” to product requirements

The EU AI Act is phasing in obligations between 2025 and 2027, with general-purpose AI obligations already applicable and most broader rules applying from August 2, 2026. NIST's AI RMF and Generative AI Profile likewise push organizations toward documentation, testing, monitoring, and risk controls. For ClashRoom, that means keeping decision logs, review trails, content rationale, and a clean line between a model suggestion and a product decision.

4. App distribution will keep tightening around privacy, clarity, and manipulated media

Apple requires privacy disclosures for data collection and expects apps to perform as advertised, while Google Play policy explicitly bars deceptive behavior and restricts manipulated media that may mislead on sensitive events, politics, social issues, or public concern. ClashRoom must expect closer scrutiny of moderation, user-generated content, media labeling, and privacy claims. What gets stored, what gets disclosed, and how “verified” language is presented must be precise.

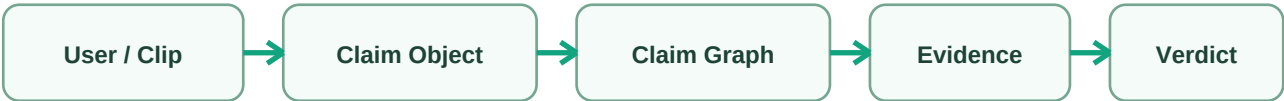
5. Trust in digital media will get harder, not easier

Content provenance standards such as C2PA are maturing, but the broader platform ecosystem is still inconsistent. That means ClashRoom should not assume the rest of the internet will hand it trustworthy provenance metadata. Instead, ClashRoom should store its own evidence package: source URL, retrieval timestamp, publisher, quote snippet, screenshot or hash where appropriate, and the path by which a verdict was reached.

Durable architecture principle

If the future is fragmented, your architecture must be modular

Do not bake any provider assumptions into core product objects. Claims, evidence, debates, and trust events are permanent business assets. Providers, prompts, embeddings, and ranking recipes are replaceable implementation details.



AI may assist extraction, search, ranking, and summaries - but the underlying records must survive without AI.

Part II - Future-proofing design rules for ClashRoom

A. Keep claims as first-class objects

A post or video is not the deepest unit of truth analysis. A claim is. Every claim must persist independently of the post that surfaced it so it can be challenged, linked, updated, verified, and reused later.

B. Keep evidence and provenance permanent

Never store only the verdict. Store the evidence list, retrieval source, timestamps, and rationale. If a model disappears, the evidence package still lets a human reviewer continue the work.

C. Build a provider router

All model calls should go through a thin provider abstraction layer. Inputs, outputs, failures, costs, latency, and safety flags should be normalized. Swapping providers should be configuration work, not a rewrite.

D. Maintain a human-operable fallback mode

Every critical workflow must have a manual path: claim creation, claim linking, evidence submission, dispute review, moderation escalation, and verdict application.

E. Use AI by task class, not by brand

Select tools by job: extraction, dedupe, search ranking, summarization, moderation triage, abuse detection, and user support. This avoids building the company around one model vendor’s strengths.

F. Add observability and policy logging from day one

Track which provider, version, prompt family, retrieval source, and reviewer state produced each suggestion. This is necessary for debugging, compliance, and credibility.

G. Preserve portability of embeddings and indexes

Vector indexes, fingerprints, and claim-similarity systems should be rebuildable from canonical text plus metadata. Do not lock the graph into one vendor-specific embedding forever.

H. Design for staged autonomy

Today: AI assists. Tomorrow: AI routes and drafts. Final authority for sensitive or public-interest claims should remain reviewable. This helps with trust, regulation, and app-store scrutiny.

Risk	What breaks	Resilient design response
Vendor disappears or degrades	Verification speed, summaries, moderation	Swap provider, use cached evidence, fall back to human review queues
API price spike	Unit economics of high-volume verification	Tiered model routing, batching, caching, premium escalation only for value
New regulation	Opaque automated decisions become risky	Decision logs, human override, disclosure language, review state machine
App-store rejection	Distribution blocked	Precise privacy labels, manipulated-media safeguards, transparent claim
Misinformation wave or synthetic media flood	Unclear media provenance	Provenance package, evidence scoring, source-class weighting, contradiction
Future dev team inherits partial knowledge or forks badly	Project knowledge	Master spec, repo map, schema docs, runbooks, architecture decision re

Part III - ClashRoom master system specification

Mission and product definition

ClashRoom is a social platform for structured debate and real-time claim verification. The product treats claims as first-class objects linked to evidence, debate threads, users, verification status, and credibility history. Its goal is to make truth socially rewarding and make misinformation challengeable. AI helps, but the product must remain operable without it.

Core permanent product assets

- Claim objects and their revisions
- Evidence records and provenance metadata
- Claim relationships: supports, contradicts, duplicates, clarifies, updates
- Trust and reputation events over time
- Moderation and review logs
- Source retrieval history and audit trails

Primary subsystems

Subsystem	Purpose	Must survive without AI?
ClashClips	Home feed of video-based debates and claim context	Yes
Claim System	Stores claims as durable objects	Yes
Evidence System	Stores sources, snippets, provenance, and scores	Yes
ClaimDNA	Detects semantic duplicates and related claim variants	Yes - manual linking fallback required
TrustPulse	Tracks credibility and evidence-weight events	Yes
ClashBot	Automates extraction, search, summarization, and draft verdicts	No - but product must still function when it is absent

Claim graph architecture

The core innovation is the claim graph. Claims should exist independently and connect to each other by relationship type. This turns the product from a feed into a reusable truth infrastructure layer.

Example relationship types

Relationship type	Meaning
supports	Claim A offers support for Claim B
contradicts	Claim A disputes Claim B
duplicates	Claim A is materially the same as Claim B
clarifies	Claim A narrows or explains Claim B
updates	Claim A supersedes Claim B with newer information

Data model blueprint

- users:** id, username, profile metadata, clashcred_score, created_at, status
- clips:** id, creator_id, video_url, caption, created_at, moderation_state
- claims:** id, text, canonical_text, creator_id, clip_id, language, status, created_at, updated_at
- evidence:** id, claim_id, url, publisher, source_type, excerpt, retrieval_time, credibility_score, submitted_by
- claim_relationships:** id, claim_a, claim_b, relationship_type, confidence, created_by, created_at
- verification_runs:** id, claim_id, provider, model_version, prompt_family, output_summary, suggested_verdict, cost, latency_ms, created_at
- trustpulse_events:** id, user_id, claim_id, event_type, score_delta, explanation, created_at
- moderation_actions:** id, object_type, object_id, action, reason, actor_type, actor_id, created_at

Verification pipeline

- AI-assisted path:** claim posted -> extraction -> search/retrieval -> evidence ranking -> suggested verdict -> human/community confirmation.
- AI-independent path:** claim posted -> manual challenge -> evidence submission -> moderator or community review -> verdict.

Rule
Never let a model output be the only durable record. The durable record is the claim, the evidence package, and the review state.

ClaimDNA and TrustPulse

ClaimDNA should identify duplicate and near-duplicate claims by combining canonicalized text, embeddings, fingerprints, and manual review tools. TrustPulse should record credibility changes as explicit events rather than as a mysterious score. A future team must be able to audit why a score changed, not just see that it changed.

Current known mobile repo snapshot

Directory / file group	Notes
app/(tabs)	Main mobile navigation and tab screens
components/clashbot	UI and interaction components related to ClashBot
components/ui	Reusable UI primitives
lib/clashbot	Providers, selectors, claimDna.ts, extractClaims.ts, verify.ts, knownFacts.ts, mockStream.ts, types.ts,
app.config.js, babel.config.js, eas.json	App configuration and build settings
package files, README.md, start-dev.ps1	Package management, docs, local startup tooling
android, .expo, app-example, backups	Platform/build artifacts and safety copies

Part IV - Human developer handoff plan

If AI vanished or this collaboration stopped tomorrow, a human team should still be able to continue. That requires documentation, not faith.

1. Repository readme overhaul

The repo needs a top-level README that explains local setup, architecture, environment variables, what is production-ready, what is mocked, and what is planned but not yet built.

2. Architecture decision records (ADRs)

Capture why the claim graph exists, why evidence is permanent, why provider abstraction matters, and what tradeoffs were accepted.

3. Schema contract doc

Create a versioned schema reference for every table and event type. Include allowed enums and state transitions.

4. API contract doc

Define request/response shapes for claim creation, evidence submission, verification triggers, verdict updates, moderation actions, and feed retrieval.

5. Runbooks

Document how to rotate providers, disable a feature flag, rebuild indexes, investigate bad verdicts, and recover from abuse spikes.

6. Testing strategy

Define unit tests for parsers and scoring, integration tests for provider routing, and golden examples for claim linking and verdict review.

7. Cost controls

Track provider spend per workflow. Add hard limits, graceful degradation, and low-cost fallback behavior.

8. Security and privacy

Document secrets handling, data retention, user deletion flow, and evidence-storage rules. Keep what you can defend.

Continuity artifact	Owner	Priority
Master spec (this document)	Founder	Complete first
Top-level README + setup guide	Engineering	Critical
Schema and API contracts	Engineering	Critical
Provider router documentation	Engineering	Critical
Moderation and review state machine	Product + engineering	High
Evidence provenance rules	Product + trust/safety	High
Incident runbooks	Engineering	High

Backlog and roadmap by subsystem	Founder + product	High
----------------------------------	-------------------	------

Recommended operating model

A small, durable team could run ClashRoom with: one full-stack product engineer, one mobile engineer, one backend/data engineer, and one trust-and-safety/product generalist. Expensive frontier AI should be treated like outside counsel: used selectively where the value justifies the cost. The day-to-day product should rely on structured data, strong review tools, and standard software engineering.

Part V - 90-day resilience plan

Time window	Action
Days 1-15	Freeze the canonical vocabulary: claim, evidence, verdict, relationship type, trust event, moderation action.
Days 1-30	Document repo setup, environment variables, current mocks, and current blockers.
Days 15-30	Implement or harden the provider router and move model-specific logic behind interfaces.
Days 15-45	Write the schema contract and event definitions. Add migration discipline.
Days 30-60	Add logging for provider, model, cost, latency, verdict suggestion, and reviewer action.
Days 30-75	Build manual fallback screens for claim linking, evidence review, and verdict override.
Days 45-90	Create golden test cases for famous disputes and failure modes.
Days 60-90	Create a local/open-weight fallback prototype for one contained task: claim dedupe, moderation triage, or evidence c

Foundational principle

Do not optimize first for “most autonomous AI.” Optimize first for “best human-operable system with AI acceleration.” That architecture survives more futures.

Sources and forecast basis

The forecast sections above draw from official policy and technical sources available as of March 13, 2026. These sources support the directional conclusions about cost trends, model convergence, regulation, app-store constraints, and content provenance. Titles are listed in plain language so future developers can re-check them as the landscape changes.

[1] Stanford HAI, “The 2025 AI Index Report” - overall findings on investment, usage, and model performance convergence.

[2] Stanford HAI, “Technical Performance” chapter - open-weight gap narrowing, frontier convergence, smaller stronger models, and reasoning tradeoffs.

[3] Stanford HAI, “Research and Development” chapter - declining inference cost trends and rising compute requirements.

[4] European Commission, “Navigating the AI Act” and “AI Act” pages - phased applicability from 2025 to 2027.

[5] NIST, “Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile” and AI RMF resource pages.

[6] Apple Developer, “App Review Guidelines,” “App Privacy Details,” and “User Privacy and Data Use.”

[7] Google Play Developer Program Policy - deceptive behavior, manipulated media, user data, and behavior transparency policies.

[8] C2PA specification overview and C2PA “Content Credentials” materials - current open standard for content provenance.

[9] OpenAI official API pricing pages - current evidence that advanced model pricing remains tiered even as low-cost options exist.

Last word

The safest strategic bet is not to assume AI disappears. It is to assume the rules, costs, vendors, and user expectations keep changing. If ClashRoom keeps its permanent assets in structured records and keeps AI inside replaceable modules, future changes will hurt less and could even become an advantage.